# ECAPA-TDNN のモバイル向け軽量化と オンデバイス話者照合 PoC

2026 年 11 月 3 日

**概要**

SpeechBrain の学習済み ECAPA-TDNN を，前処理を含む単一の ONNX モデルへ変換し，ONNX Runtime の静的 INT8 量子化によって軽量化した．モデルサイズは 84.1 MB から 25.8 MB に減少し，CPU 上の推論時間は約 5 分の 1 となった一方，LibriSpeech 上の等誤り率（EER）に劣化は見られなかった．さらに，このモデルを用いて登録音声と発話音声の同一性を判定する Flutter アプリを試作した．

## 1　目的

話者照合モデル ECAPA-TDNN をスマートフォン上で動作させ，事前に登録した音声と新たな発話が同一人物によるものかをオンデバイスで判定することを目的とする．アプリは Flutter で実装し，推論には ONNX Runtime を用いる．

## 2　手法

### 2.1　ベースモデル

SpeechBrain が公開する `spkrec-ecapa-voxceleb`（VoxCeleb で学習，パラメータ数約 2,080 万，埋め込み 192 次元）を用いた．本報告ではモデル構造の変更や再学習は行っていない．

### 2.2　前処理を同梱した ONNX 化

元のモデルは 80 次元の対数メルフィルタバンク（Fbank）を入力とする．アプリ側で同じ特徴量抽出を再実装すると，実装差による精度劣化の原因となるため，Fbank をモデルに含め

た．具体的には，ハミング窓をかけた DFT 基底を重みとする 1 次元畳み込みで STFT を表現し，パワースペクトルへのメル行列の乗算，対数化，ダイナミックレンジ 80 dB でのクリップ，発話単位の平均除去をすべて基本演算で記述した．STFT 演算子を用いないため，古いランタイムや各種実行プロバイダでも動作しやすい．得られたモデルの入出力は次のとおりである．

- 入力：16 kHz モノラル波形（float32，長さ可変）
- 出力：L2 正規化済み 192 次元埋め込み

SpeechBrain 本来の推論結果との一致を確認したところ，コサイン類似度は 1.000000 であった．また，1.5 秒，3 秒，7 秒の入力で PyTorch と ONNX Runtime の出力差は最大 $1.9 \times 10^{-7}$ であり，可変長入力が正しく扱えることを確認した．

### 2.3 量子化

ONNX Runtime の量子化ツールを用いて次の 2 方式を比較した．

- **動的量子化**：重みのみを INT8 化し，活性のスケールは実行時に求める．校正データは不要である．
- **静的量子化（QDQ 形式）**：重みをチャネル単位で INT8 化し，活性は校正データから求めた範囲（99.99 パーセンタイル）で UINT8 化する．

いずれも前処理部分（DFT 畳み込み，対数メル）は精度保持のため FP32 のまま残した．

### 2.4 照合方式

登録時に 3 回の発話からそれぞれ埋め込みを求め，その平均を再正規化して登録声紋とする．照合時は発話の埋め込みと登録声紋のコサイン類似度 $s$ を求め，$s \geq \theta$ ならば本人と判定する．入力前にはフレームエネルギーに基づく簡易な無音除去を行い，有声区間が 1.5 秒未満の発話は棄却する．

## 3 実験

### 3.1 条件

評価には LibriSpeech を用いた．test-clean から 40 話者 ×12 発話（3 秒以上）を抽出したものに，動作確認用に取得していた dev-clean の 1 話者 73 発話が加わった計 41 話者 553 発話を用意した．このうち 10 話者を校正専用とし（各 2 発話を使用），残る 31 話者 433 発話を評価に用いた．各発話の先頭 4 秒から埋め込みを求め，全組合せ 93,528 試行（本人 4,608，他人 88,920）で EER を算出した．推論時間は x86 CPU 上で 1 スレッド，3 秒入力に対する 10 回の

平均である．

### 3.2 結果

結果を表 1 に示す．

**表 1　量子化方式ごとの比較**

| モデル | サイズ [MB] | EER [%] | EER 閾値 | FP32 との cos | 推論 [ms] |
|---|---|---|---|---|---|
| FP32 | 84.1 | 0.80 | 0.327 | 1.0000 | 225.6 |
| INT8 動的 | 22.0 | 0.78 | 0.334 | 0.9947 | 320.1 |
| INT8 静的（QDQ） | 25.8 | 0.78 | 0.329 | 0.9957 | 45.6 |

静的量子化はサイズを約 3 分の 1 に，推論時間を約 5 分の 1 にしつつ，EER を維持した．動的量子化はサイズでは最小であるが，畳み込み主体のモデルでは実行時の量子化処理が負担となり，FP32 より遅くなった．推論時間には測定ごとのばらつきがあり，別の測定ではそれぞれ 393.7，607.8，77.8 ms であったが，相対的な傾向は変わらない．以上より，アプリには静的量子化モデルを採用した．

なお，Flutter パッケージに同梱される ONNX Runtime 1.15.1 でも，FP32 モデルと静的量子化モデルが読み込めて，正規化済み埋め込みが得られることを確認した．

### 3.3 実装上の問題と対処

1. **可変長入力での破綻**：全演算子を量子化対象にすると，Attentive Statistics Pooling のマスク長計算まで量子化され，校正時と異なる長さの入力でエラーとなった．量子化対象を Conv と MatMul に限定して解決した．
2. **ランタイムのクラッシュ**：ONNX Runtime 1.31 の QDQ 融合最適化が，最終全結合層（Pad を経由する 1×1 畳み込み）でクラッシュした．当該ノード（約 120 万パラメータ）のみ FP32 で残して回避した．
3. **旧ランタイムでの読込失敗**：量子化ツールが付与する未使用の opset 宣言（`ai.onnx.ml` v5 など）により，ONNX Runtime 1.15 がロードを拒否した．未使用の宣言を削除して解決した．

## 4 Flutter アプリ

`onnxruntime` パッケージ（1.4.1）と `record` パッケージを用い，Android ／ iOS 向けに実装した．マイクから 16 kHz の PCM を取得して float に変換し，無音除去の後にモデルへ入力す

る．登録声紋は端末内に保存し，判定閾値は画面上のスライダーで調整できる．OS のノイズ抑圧や自動利得制御は声質を変化させるため無効にしている．静的解析とユニットテストは通過しているが，APK のビルドおよび実機での動作確認は未実施である．

## 5 課題と今後

- **評価の妥当性**：評価は英語の読み上げ音声によるもので，規模も小さい．日本語，実機のマイク，雑音環境での評価と閾値の再調整が必要である．
- **実機性能**：スマートフォン上での推論時間，メモリ使用量，ランタイムを含むアプリサイズを測定する必要がある．
- **さらなる軽量化**：チャネル数を 1024 から 512 に減らした構造を，再学習または知識蒸留で学習すれば，6〜8 MB 程度への縮小が見込める．WeSpeaker などの小型学習済みモデルとの比較も考えられる．
- **安全性**：録音音声の再生によるなりすましへの対策が別途必要である．

## 6 まとめ

ECAPA-TDNN を前処理込みの単一 ONNX モデルとし，静的 INT8 量子化を施すことで，精度を保ったまま大幅な軽量化と高速化を達成した．また，これを用いた Flutter によるオンデバイス話者照合の試作を行った．今後は実機と実環境の音声による評価を進める．